

DESARROLLO DE COMPONENTES

Contenido

1. Introducción.....	3
2. Modelo Vista Controlador (MVC).....	3
3. Desarrollando un módulo en Odoo.....	3
3.1. Configuración inicial	4
3.1.1. Servicio de Odoo.....	4
3.1.2. Archivo log de errores	4
3.1.3. Editor Visual Studio Code.....	5
3.1.4. IP elástica en AWS.....	9
3.1.5 Esqueleto del módulo	11
4. Creación de modelos	13
5. Creación de vistas.....	16
5.1. Vistas de tipo árbol y formulario.....	17
5.2. Acciones de ventana	18
5.3. Elementos de menú.....	18
6. Creación de relaciones.....	19
7. Creación de vistas de búsqueda	22
8. Creación de campos calculados.....	24
9. Creación de informes	26
10. Configuración de permisos y grupos	30
11. Cuestiones finales	32
11.1 Establecer icono	32
11.2. Datos precargados	33

1. Introducción

En esta unidad, en contraposición con la unidad de trabajo anterior, veremos cómo realizar modificaciones en el ERP utilizando el lenguaje de programación Python. De esta forma, desarrollaremos módulos que nos permitirán modificar el ERP de una forma automática y sencilla, simplemente instalando el módulo.

Las modificaciones que aprenderemos a realizar serán muy similares a las realizadas en la UD4, pero el procedimiento llevado a cabo para realizarlas será totalmente diferente.

2. Modelo Vista Controlador (MVC)

El modelo Vista Controlador (MVC) es un patrón de arquitectura de software que divide una aplicación en tres componentes:

- Los datos de la aplicación (modelo).
- La interfaz del usuario (vista).
- El controlador, el cual define la forma en que la interfaz reacciona a la entrada del usuario.

Con esto se consigue separar los datos (modelo) de la interfaz de usuario (vista), de manera que los cambios en la interfaz no afectan a los datos y viceversa, es decir, que los datos pueden ser cambiados sin afectar a la interfaz de usuario.

En Odoo, el MVC se implementa de la siguiente forma:

- El modelo son las tablas de la base de datos. Vienen representadas por una clase en Python, que al instanciarse es un objeto en Odoo.
- Las vistas son los archivos XML que definen la interfaz de usuario del módulo.
- El controlador son los objetos creados en Python. En módulos sencillos esta capa puede no ser necesaria.

3. Desarrollando un módulo en Odoo

El primer concepto que se debe tener claro es que un módulo es un directorio dentro de la carpeta **addons** de nuestro servidor. Este directorio tendrá una serie de archivos que definirán el módulo.

Es vital que sepamos realizar las siguientes tareas antes de comenzar con el desarrollo del módulo:

- Ser capaces de arrancar y parar el servicio de Odoo. Necesitaremos hacerlo para que se recarguen los archivos Python de nuestro módulo.
- Ser capaces de editar archivos en remoto en el servidor. Para ello necesitaremos un editor que pueda trabajar sobre SFTP y que nos facilite la tarea. En este curso utilizaremos **Visual Studio Code**, pero se pueden emplear otros muchos editores.

3.1. Configuración inicial

Para poder desarrollar un nuevo módulo desde cero, es necesario realizar una serie de configuraciones previas.

Como ya sabemos, la aplicación Odoo se encuentra instalada en la ruta `/home/odoo/odoo` del servidor, en la cual está ubicado el directorio **addons**, que contiene todos los módulos candidatos para la instalación. El módulo que se va a crear en esta unidad va a ser otro directorio dentro de esta carpeta **addons**, que contendrá todos los objetos, vistas, informes y demás.

3.1.1. Servicio de Odoo

En cuanto a tareas antes de comenzar el desarrollo, como ya se dijo antes es necesario saber cómo reiniciar el servicio de Odoo.

`sudo systemctl restart odoo.service`

Esta operación será necesario realizarla cuando se hagan cambios en los archivos de Python del módulo, para que se recarguen.

3.1.2. Archivo log de errores

Otro aspecto que debemos considerar es el tema de los errores. En el caso de que se cometa algún error en un archivo de Python puede ser que el servidor no arranque, es decir, que el archivo **odoo.service** dé un error. Para poder consultar dicho error es necesario ver el archivo de **log** donde Odoo guarda los errores. Recordemos que el archivo de servicio de Odoo (**odoo.service**) se encuentra en la ruta `/etc/systemd/system`. Para poder ver el archivo log donde se guardan los errores, no hay más que añadir en el archivo **odoo.service** la ruta del archivo log.

```
GNU nano 4.8                                odoo.service
[Unit]
Description=Odoo Open Source ERP and CRM
After=network.target

[Service]
Type=simple
User=odoo
Group=odoo
ExecStart=/home/odoo/odoo/odoo-bin -c /home/odoo/odoo-server.conf --logfile /home/odoo/odoo.log
KillMode=mixed

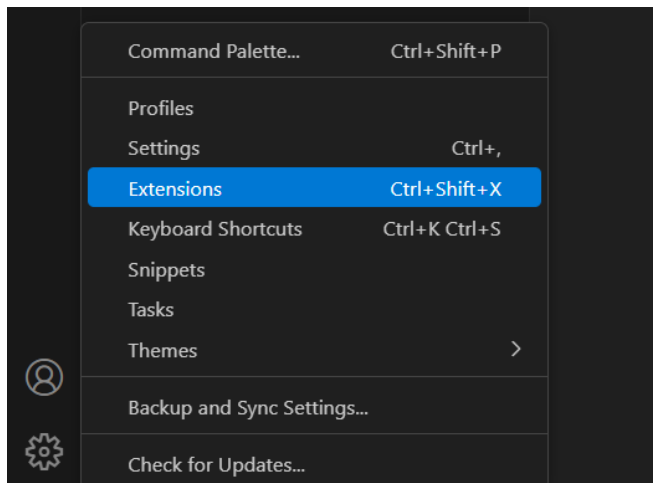
[Install]
WantedBy=multi-user.target
```

Una vez ya vistos estos dos aspectos, se comienza a crear el módulo. Para ello bastaría con crear un nuevo directorio dentro de **addons** e ir editando archivos, con el editor nano o con el editor pico. Sin embargo, esto implicaría hacerlo todo mediante consola. Para evitar esto, lo mejor es utilizar un editor externo.

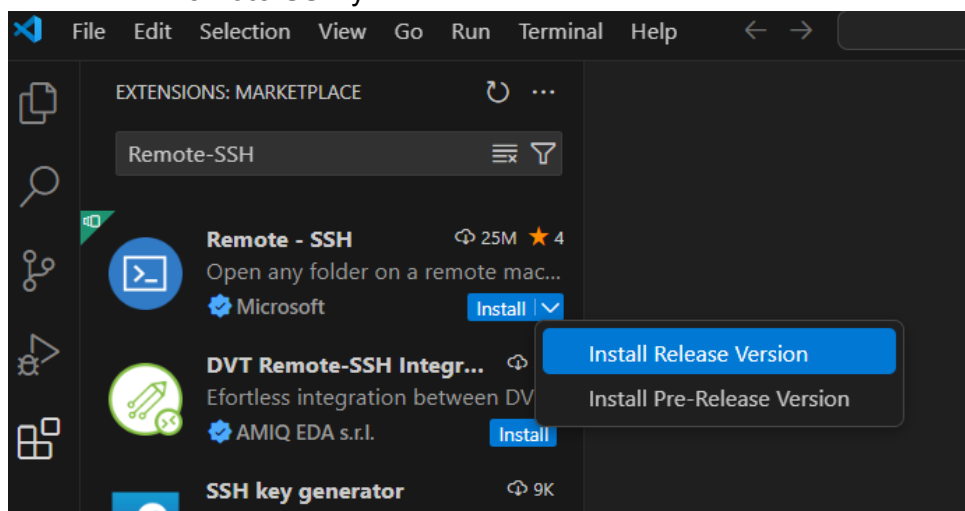
3.1.3. Editor Visual Studio Code

Esta herramienta permite editar archivos en remoto con el protocolo SFTP, que es lo que necesitamos para poder editar archivos en una máquina virtual desde fuera de la máquina. Una vez instalado el editor, se debe instalar también una extensión (Remote-SSH) para poder editar archivos en remoto. Para ello se hace lo que indica en la imagen.

Desde Ajustes seleccionamos **extensiones**:



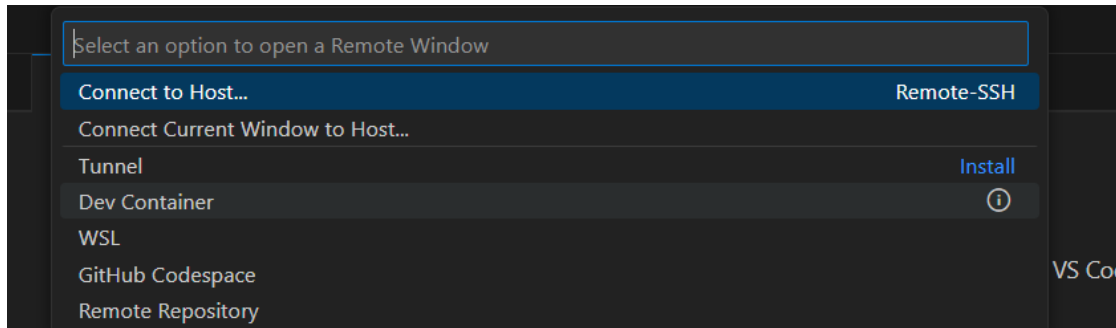
Luego buscamos **Remote-SSH** y le damos a instalar:



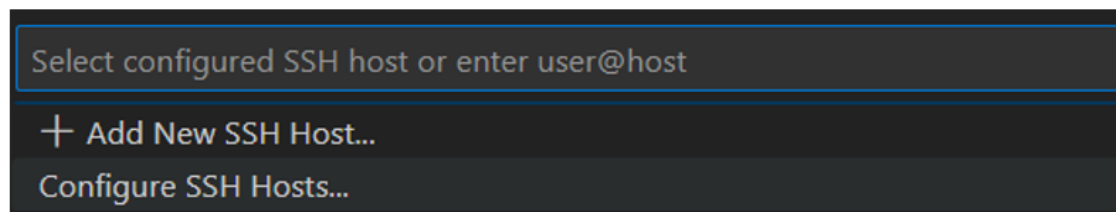
Una vez instalado tenemos que rellenar los datos del servidor. Pinchamos en el cuadrado azul de la barra inferior, a la izquierda



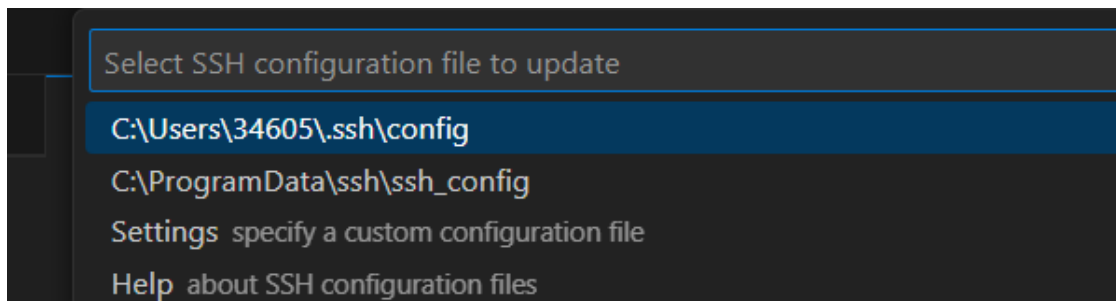
En la parte superior de la ventana seleccionamos **Connect to Host...**



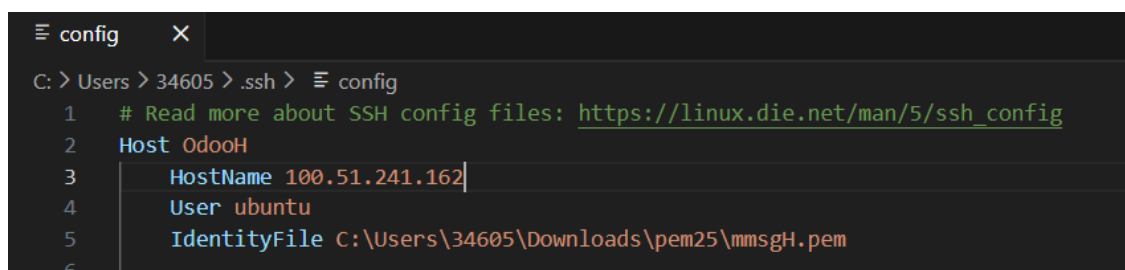
Y luego **Configure SSH Host...**



Nos muestra la ruta al archivo de configuración que debemos abrir en nuestro usuario local de Windows:



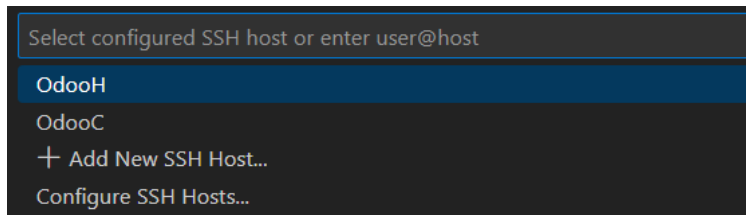
Lo editamos con los datos referentes al servidor remoto que tenemos en AWS:



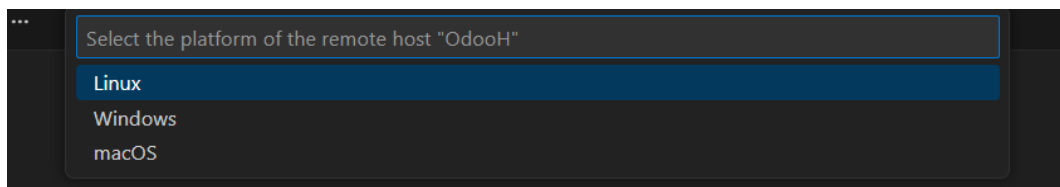
- Host: Nombre del alias del servidor al que nos queremos conectar.
- Hostname: Dirección IP pública de nuestra instancia en AWS

- Usuario: ubuntu
- IdentityFile: Ruta a nuestro archivo .pem para realizar la conexión ssh.

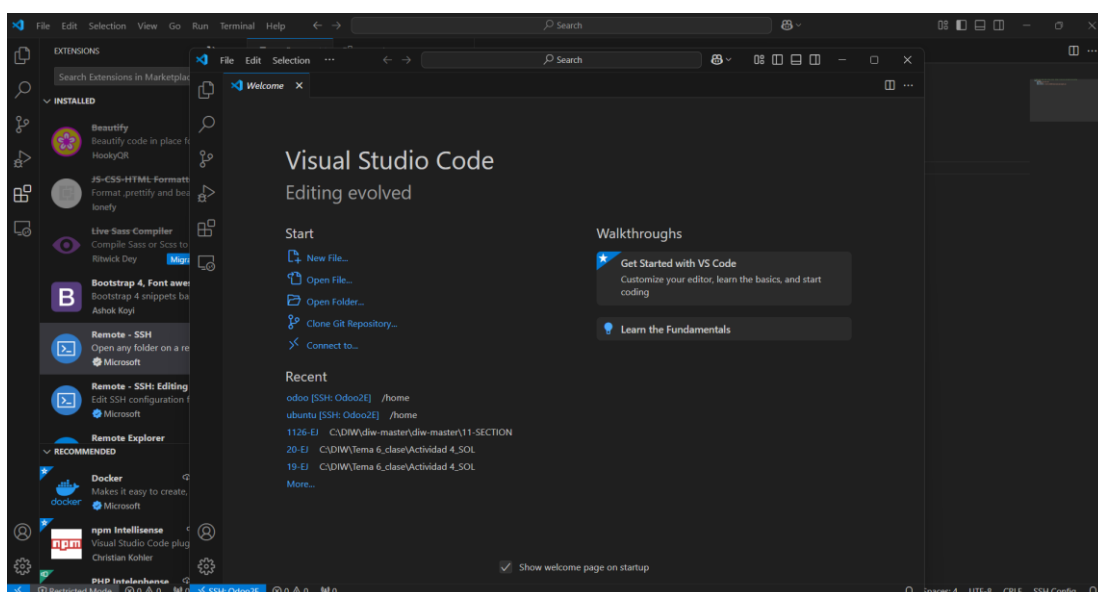
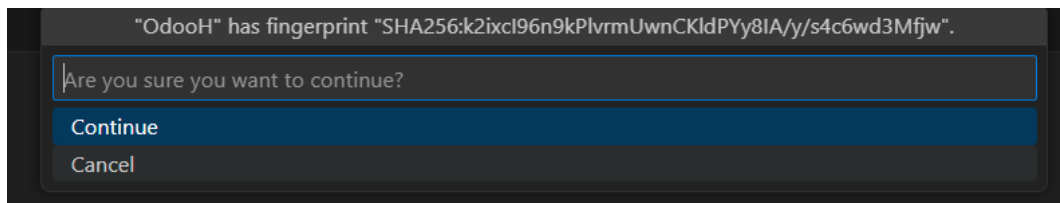
Guardamos el archivo y nos aparecerá el servidor disponible para conectar cuando volvamos a Connect to Host...



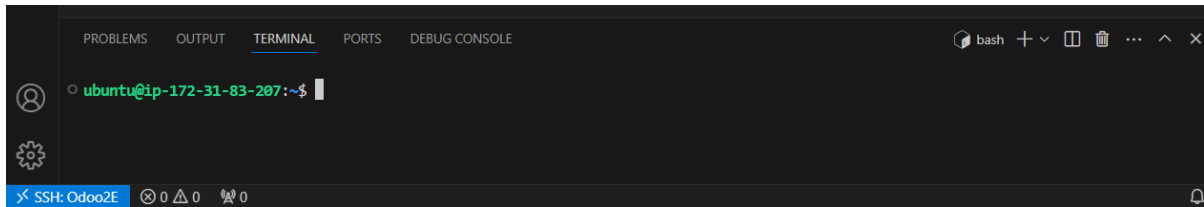
Lo seleccionamos. Se abre una nueva ventana, que nos preguntará el tipo de servidor al que nos queremos conectar (Linux, Windows o macOS)



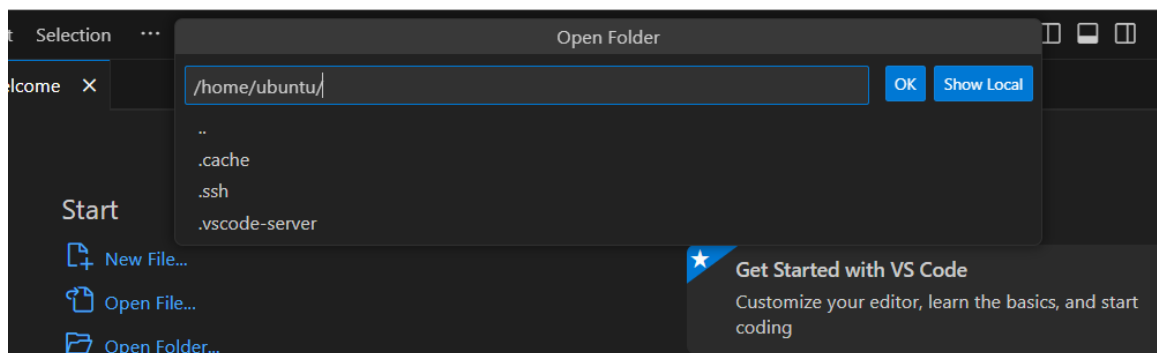
luego nos pedirá confirmación para continuar y si todo va bien nos conectaremos con el servidor:



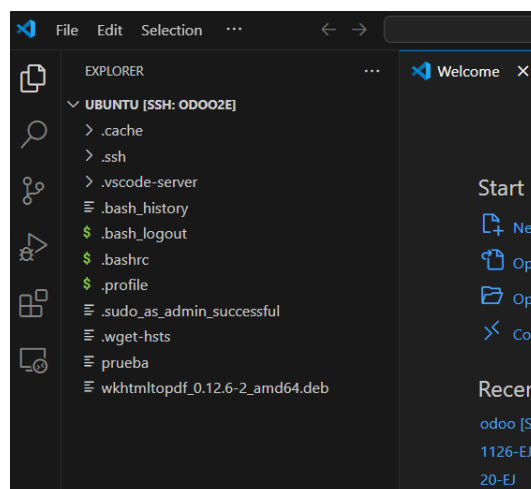
Si mostramos el terminal veremos que estamos en la carpeta principal del usuario Ubuntu



Si pulsamos en open folder nos muestra las rutas que ve:

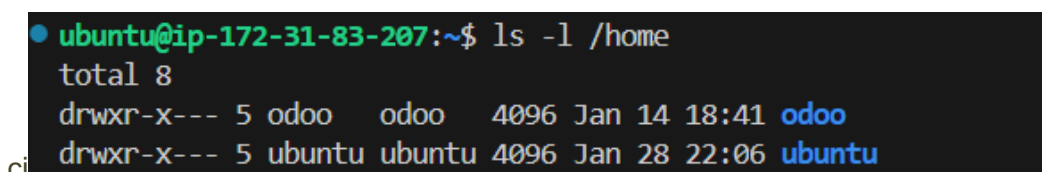


Si aceptamos visualizaremos el contenido en el explorador de la izquierda:



En realidad, queremos acceder al directorio addons que es donde se sitúan los módulos.

Esa carpeta pertenece al usuario odoo y necesitamos permisos para poder acceder a ella. El usuario con el que nos conectamos es ubuntu, que no tiene acceso a la carpeta odoo pero se lo vamos a dar:



Primero hacemos que los usuarios del grupo odoo tengan todos los permisos en la carpeta principal y luego añadimos al usuario ubuntu a ese grupo.

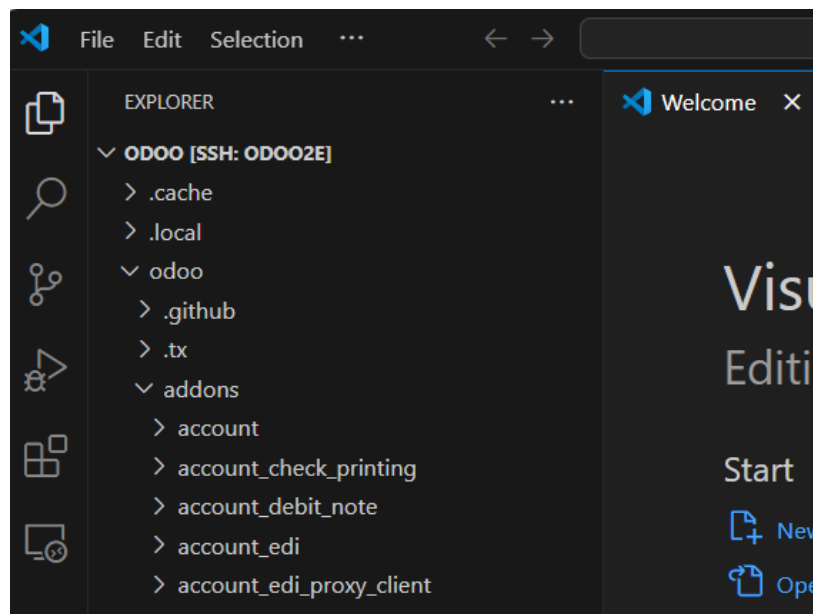
```
● ubuntu@ip-172-31-83-207:~$ sudo chmod 770 /home/odoo
● ubuntu@ip-172-31-83-207:~$ ls -l /home
total 8
drwxrwx--- 5 odoo odoo 4096 Jan 14 18:41 odoo
drwxr-x--- 5 ubuntu ubuntu 4096 Jan 28 22:06 ubuntu
```

```
● ubuntu@ip-172-31-83-207:~$ sudo adduser ubuntu odoo
Adding user `ubuntu' to group `odoo' ...
Adding user ubuntu to group odoo
Done.
```

Para que las modificaciones de permisos tengan efecto hay que volver a hacer login. En este caso puede que incluso tengamos que reiniciar la instancia, pero finalmente veremos que podemos acceder al directorio /home/odoo y a lo que cuelga de él.

Podemos probar primero en el terminal y una vez que el acceso va bien conectarnos a la carpeta para verla en el explorador lateral.

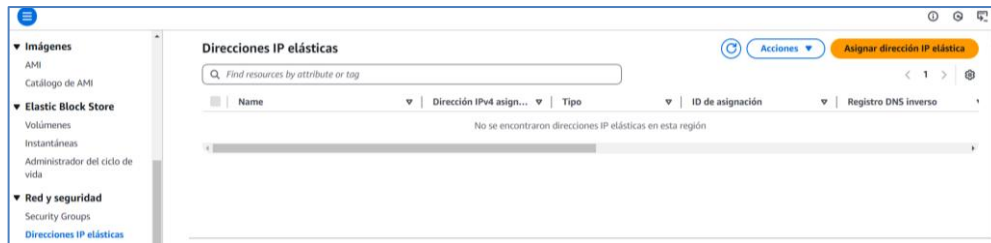
Ahora ya podemos conectarnos a ese directorio para acceder a la carpeta addons.



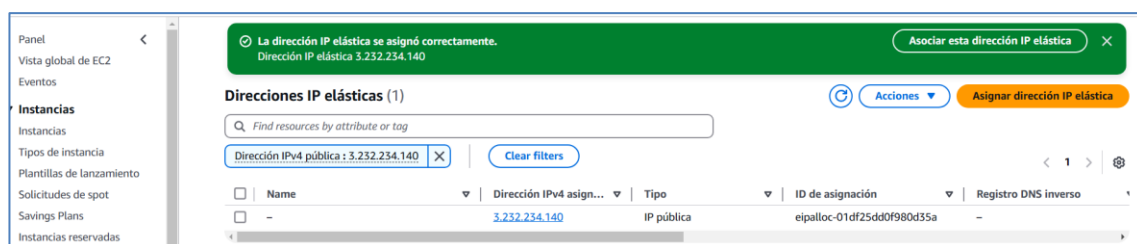
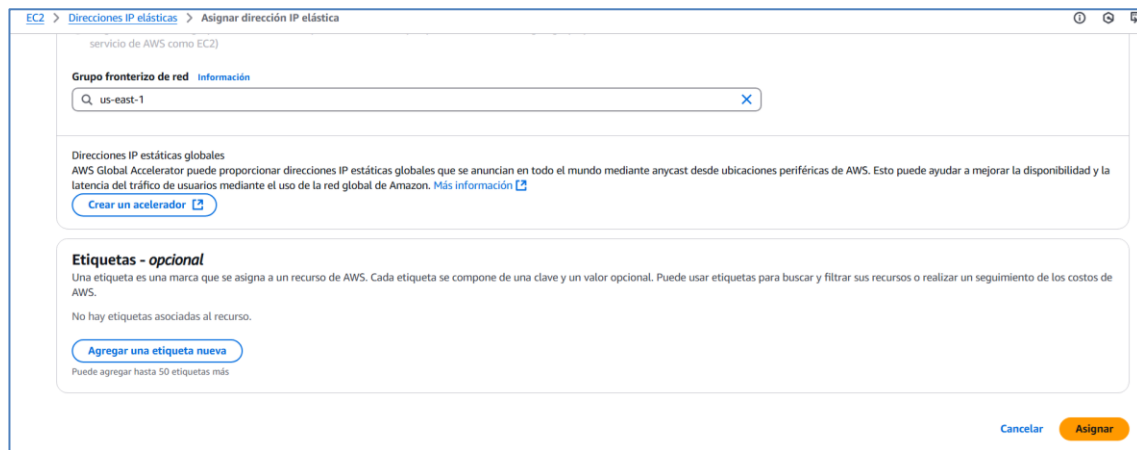
3.1.4. IP elástica en AWS

Como sabemos, cada vez que inicializamos nuestra instancia del servidor en AWS la IP puede cambiar. Como en esta unidad nos vamos a conectar al servidor a través de Visual Studio Code, si queremos que la configuración se mantenga, podemos fijar la IP configurando una IP elástica en nuestro servidor.

En el menú de la izquierda de la instancia seleccionamos en el bloque Red y Seguridad > Direcciones IP elásticas. Luego, como siempre botón naranja, “Asignar dirección IP elástica”



Luego asignar



Asociar esta dirección IP elástica en el botón verde y seleccionar la instancia a la que se la queremos asignar

EC2 > Direcciones IP elásticas > Dirección IP elástica asociada

Dirección IP elástica: 3.232.234.140

Tipo de recurso
Elija el tipo de recurso al que desea asociar la dirección IP elástica.

☒ Instancia
☐ Interfaz de red

⚠ Si asocia una dirección IP elástica a una instancia que ya tiene una dirección IP elástica asociada, la dirección IP elástica asociada anteriormente se desasociará, pero la dirección seguirá asignándose a su cuenta. [Más información](#)

Si no se especifica ninguna dirección IP privada, la dirección IP elástica se asociará a la dirección IP privada principal.

Instancia
Fija una instancia

i-003891b689da6351c (Odoo2E) - running

Elija una dirección IP privada

Reasociación
Especifique si la dirección IP elástica se puede volver a asociar a un recurso diferente si ya está asociada a un recurso.

☒ Permitir que se vuelva a asociar esta dirección IP elástica

Cancelar Asociado

Dejar asociado para que permanezca fijo y comprobar que es la que sale en las propiedades de la instancia. Ya será siempre esa IP por lo que podemos configurar el host con esa y no habrá que modificarlo.

Instancias (1/1) Información						Última actualización Hace less than a minute	Conectar	Estado de
Buscar Instancia por atributo o etiqueta (case-sensitive) Todos los ...								
☒	Name	ID de la instancia	Estado de la i...	Tipo de inst...	Comprobación c			
☒	Odoo2E	i-003891b689da6351c	En ejecución	t2.micro	2/2 comprob			
i-003891b689da6351c (Odoo2E)								
Detalles Estado y alarmas Monitoreo Seguridad Redes Almacenamiento Etiquetas								
Resumen de instancia Información								
ID de la instancia i-003891b689da6351c			Dirección IPv4 pública 3.232.234.140 dirección abierta					

3.1.5 Esqueleto del módulo

A continuación, se va a crear el directorio del nuevo módulo. Será un módulo llamado *libreria* que tendrá dos tablas: categorías y libros.

Para ello se utiliza un comando que crea automáticamente el esqueleto del nuevo módulo:

- En el servidor, se va al directorio donde está el odoo-bin (/home/odoo/odoo).

```
profesor@ubuntuserver:/home/odoo/odoo/addons$ cd ..
profesor@ubuntuserver:/home/odoo/odoo$ ls
CONTRIBUTING.md  LICENSE      README.md    addons  doc  odoo-bin  setup  setup.py
COPYRIGHT        MANIFEST.in SECURITY.md  debian  odoo  requirements.txt  setup.cfg
profesor@ubuntuserver:/home/odoo/odoo$ pwd
/home/odoo/odoo
profesor@ubuntuserver:/home/odoo/odoo$
```

- Se ejecuta el siguiente comando (recordad, desde el directorio /home/odoo/odoo);

`su odoo`

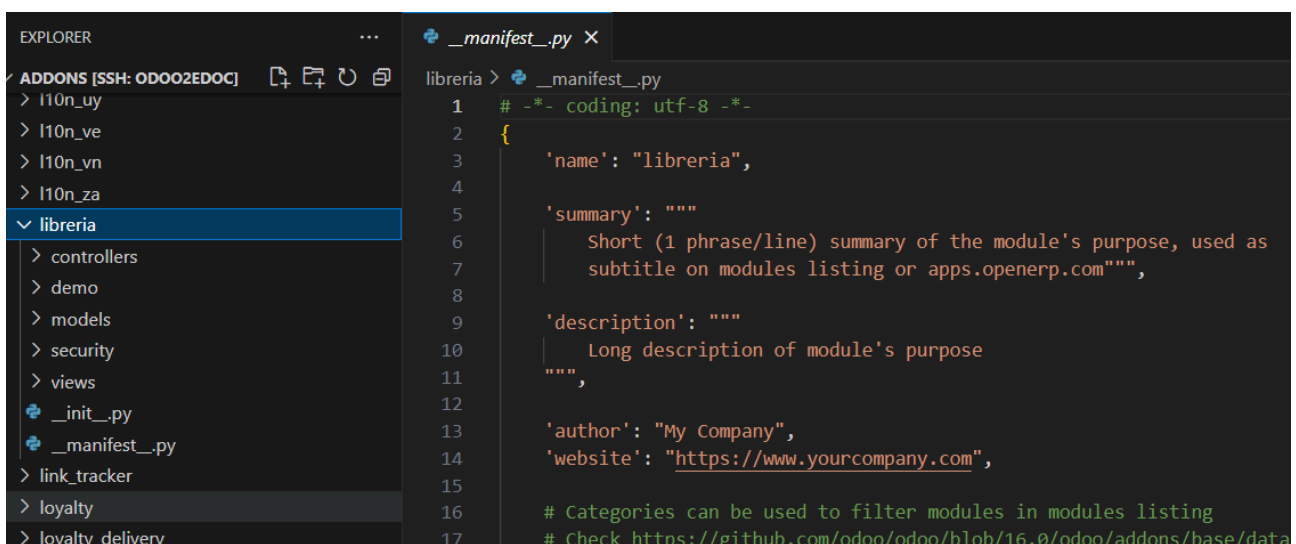
`./odoo-bin scaffold [nombre-módulo] [ruta-relativa--addons]`, quedando como sigue:

`./odoo-bin scaffold libreria addons`

NOTA: Se debe hacer desde un usuario con permisos, el usuario odoo o el usuario Ubuntu, una vez que se los hemos concedido.

```
ubuntu@ip-172-31-83-207:/home/odoo/odoo$ su odoo
Password:
odoo@ip-172-31-83-207:~/odoo$ ./odoo-bin scaffold libreria addons
odoo@ip-172-31-83-207:~/odoo$
```

Si ahora entramos en el addons desde el servidor, ya se podrá ver el directorio libreria con su estructura, para poder empezar a trabajar con él.



```
EXPLORER
ADDONS [SSH: ODOO2EDOC]
  > l10n_uy
  > l10n_ve
  > l10n_vn
  > l10n_zs
  ▼ libreria
    > controllers
    > demo
    > models
    > security
    > views
    > __init__.py
    > __manifest__.py
    > link_tracker
    > loyalty
    > loyalty_delivery

libreria > __manifest__.py
1  # -*- coding: utf-8 -*-
2  {
3      'name': "libreria",
4
5      'summary': ""
6          Short (1 phrase/line) summary of the module's purpose, used as
7          subtitle on modules listing or apps.openerp.com"",
8
9      'description': ""
10         Long description of module's purpose
11         "",
12
13      'author': "My Company",
14      'website': "https://www.yourcompany.com",
15
16      # Categories can be used to filter modules in modules listing
17      # Check https://github.com/odoo/odoo/blob/16.0/odoo/addons/base/data
```

En cuanto a los archivos que constituyen la estructura, señalar:

- `__init__.py`: es el primer archivo Python en abrirse, e indica qué otros archivos Python se deben importar.
- `__manifest__.py`: es el archivo que describe el módulo. Es un diccionario, con una serie de claves y valores, que va a servir a Odoo para rellenar la página de Aplicaciones con la

descripción del nuevo módulo. Se edita este archivo desde VSC dejándolo como se indica:

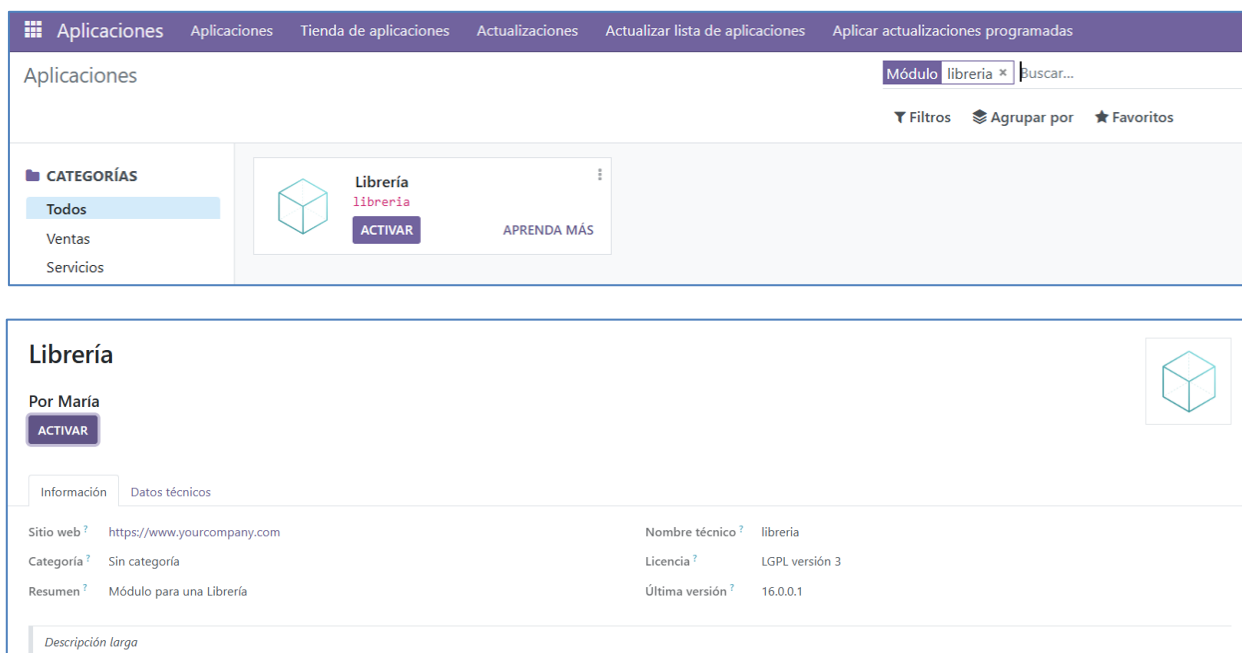
```

1  # -*- coding: utf-8 -*-
2
3  'name': "Librería",
4
5  'summary': ""
6  |  Módulo para una Librería"",
7
8  'description': ""
9  |  Descripción larga"",
10
11  'author': "María",
12  'website': "https://www.yourcompany.com",
13
14  # Categories can be used to filter modules in modules listing
15  # Check https://github.com/odoo/odoo/blob/16.0/odoo/addons/base/data/ir_module_category_data.xml
16  # for the full list
17  'category': 'Uncategorized',

```

Se pusieron datos en: Nombre, Descripción corta, Descripción larga y Autor

Salvamos con Ctrl+s, y si ahora vamos desde el navegador a Odoo y actualizamos la lista de aplicaciones (para lo cual se debe tener activado el modo desarrollador), se debería ver que el nuevo módulo libreria ya aparece.



Este módulo está vacío, es decir, no haría nada si se instalara. Ahora debe ir rellenándose.

4. Creación de modelos

Los modelos van a corresponder con tablas de base de datos y son el fundamento de nuestro módulo. En este ejemplo, vamos a realizar un módulo para una *librería*. Nuestros objetos modelo serán *libro* y *categoría*. Los archivos de modelo se escriben en Python y siguen la arquitectura que

nos proporciona Odoo.

Como lleva un modelo Vista Controlador (MVC), el modelo es lo que corresponde a las tablas de la base de datos, y los modelos van declarados en el directorio **models**, dentro del archivo **models.py**. Se deben tener en cuenta las siguientes consideraciones:

- Cada objeto en Odoo corresponde con una clase Python. En este caso, tendremos dos objetos, categoría y libro, luego se deben crear dos clases. Para nombrarlas, lo mejor es utilizar la nomenclatura *nombremodulo_nombreobjeto*.
- Todas las clases en Odoo extienden de la clase **Model** de ORM que nos proporciona Odoo. ORM es el acrónimo de *Object Relational Mapping*, o mapeo objeto-relacional, y tiene como objetivo la manipulación de los datos de una base de datos relacional a través de la manipulación de objetos sencillos.
- Todas las clases en Odoo tienen un campo privado **_name**, que se suele nombrar como *nombremodulo.nombreobjeto*.
- Además, existe otro campo **name** que ya se podrá editar desde las vistas.
- Cada campo puede tener una serie de atributos, entre ellos, la etiqueta que sale en la vista, si es requerido o no (si no es requerido, no hace falta poner nada), una ayuda para mostrar al situar el ratón encima del campo, un valor por defecto, etc.

Por otra parte, se debe señalar que los archivos que se importan al principio del archivo (models, fields, api) se encuentran en la siguiente ruta del servidor:

```
odoo@ip-172-31-83-207:~/odoo$ cd odoo
odoo@ip-172-31-83-207:~/odoo/odoo$ ls -lht
total 784K
drwxrwxr-x 4 odoo odoo 4.0K Jan 14 18:18 cli
drwxrwxr-x 3 odoo odoo 4.0K Jan 14 18:18 tests
drwxrwxr-x 2 odoo odoo 4.0K Jan 14 18:18 __pycache__
drwxrwxr-x 3 odoo odoo 4.0K Jan 14 18:18 service
drwxrwxr-x 3 odoo odoo 4.0K Jan 14 18:18 osv
drwxrwxr-x 6 odoo odoo 4.0K Jan 14 18:18 tools
drwxrwxr-x 3 odoo odoo 4.0K Jan 14 18:18 modules
drwxrwxr-x 3 odoo odoo 4.0K Jan 14 18:18 conf
drwxrwxr-x 33 odoo odoo 4.0K Jan 14 18:18 addons
drwxrwxr-x 3 odoo odoo 4.0K Jan 14 18:18 upgrade
-rw-rw-r-- 1 odoo odoo 1.7K Jan 13 21:42 release.py
-rw-rw-r-- 1 odoo odoo 30K Jan 13 21:42 sql_db.py
-rw-rw-r-- 1 odoo odoo 302K Jan 13 21:42 models.py
-rw-rw-r-- 1 odoo odoo 13K Jan 13 21:42 netsvc.py
-rw-rw-r-- 1 odoo odoo 219K Jan 13 21:42 fields.py
-rw-rw-r-- 1 odoo odoo 80K Jan 13 21:42 http.py
-rw-rw-r-- 1 odoo odoo 14K Jan 13 21:42 import_xml.rng
-rw-rw-r-- 1 odoo odoo 3.6K Jan 13 21:42 loglevels.py
-rw-rw-r-- 1 odoo odoo 3.7K Jan 13 21:42 exceptions.py
-rw-rw-r-- 1 odoo odoo 53K Jan 13 21:42 api.py
-rw-rw-r-- 1 odoo odoo 5.5K Jan 13 21:42 __init__.py
odoo@ip-172-31-83-207:~/odoo/odoo$
```

Así, la definición de ambas clases queda como se indica a continuación:

```

1  # -*- coding: utf-8 -*-
2
3  from odoo import models, fields, api
4
5  class libreria_categoria(models.Model):
6      _name = "libreria_categoria"
7
8      name = fields.Char(string="Nombre", required=True, help="Introduce el nombre de la categoria")
9      description = fields.Text(string="Descripción")
10
11
12  class libreria_libro(models.Model):
13      _name = "libreria_libro"
14
15      name = fields.Char(string="Titulo", required=True)
16      precio = fields.Float(string="Precio")
17      ejemplares = fields.Integer(string="Ejemplares")
18      fecha = fields.Date(string="Fecha de compra")
19      segundo_mano = fields.Boolean(string="Segunda mano")
20      estado = fields.Selection([('0', 'Bueno'), ('1', 'Regular'), ('2', 'Malo')], string="Estado", default="0")
21
22
23  # class libreria(models.Model):
24  #     _name = 'libreria.libreria'

```

Al salvar con **Ctrl+s**, y debido a que esto que acabamos de modificar es un archivo Python, se tiene que reiniciar el servidor.

```

● ubuntu@ip-172-31-83-207:/home/odoo/odoo$ sudo systemctl restart odoo.service
○ ubuntu@ip-172-31-83-207:/home/odoo/odoo$

```

Si ahora entramos en la aplicación Odoo, como siempre lo primero sería Actualizar la lista de aplicaciones, y después instalamos el módulo libreria, se comprueba que no tiene menús, ni vistas ni nada, porque todavía no ha sido creado, pero se puede ver que ya existen dos objetos que representan dos modelos en la estructura de la base de datos.

De igual forma, se puede comprobar que existen dos tablas más desde el pgAdmin que corresponden a los dos modelos nuevos creados, es decir, el modelo categoria, con nombre de la clase *libreria_categoria*, y el modelo libro, con nombre de la clase *libreria_libro*.

Por último, al igual que cuando se crea una nueva entidad desde la interfaz, se debe especificar qué grupo o grupos van a tener **permiso de acceso** sobre estos nuevos modelos creados.

Modelo	Descripción del modelo	Tipo	Modelo transitorio
☐ libreria_categoria	libreria_categoria	Objeto base	☐
☐ libreria_libro	libreria_libro	Objeto base	☐

Modelos / libreria.categoria

Descripción del modelo [?] libreria.categoria Tipo [?] Objeto base

Modelo [?] libreria.categoria En las aplicaciones [?] libreria

Pedido [?] id

Modelo transitorio [?] ☐

Tiene un hilo de correos [?] ☐

Tiene actividad de correo [?] ☐

Tiene lista negra de correo [?] ☐

Campos Permisos de acceso Reglas de registro Notas Vistas

Nombre	Grupo	Acce...	Per...	Crea...	Elim...
UsuarioInterno	Tipos de usuario / Usuario interno	☒	☒	☒	☒

Añadir una línea

Modelos / libreria.libro

Descripción del modelo [?] libreria.libro Tipo [?] Objeto base

Modelo [?] libreria.libro En las aplicaciones [?] libreria

Pedido [?] id

Modelo transitorio [?] ☐

Tiene un hilo de correos [?] ☐

Tiene actividad de correo [?] ☐

Tiene lista negra de correo [?] ☐

Campos Permisos de acceso Reglas de registro Notas Vistas

Nombre	Grupo	Acce...	Per...	Crea...	Elim...
UsuarioInterno	Tipos de usuario / Usuario interno	☒	☒	☒	☒

Añadir una línea

pgAdmin File Object Tools Help

Browser

- > ir_ui_view_custom
- > ir_ui_view_group_rel
- > **libreria_categoria**
 - > Columns
 - > Constraints
 - > Indexes
 - > RLS Policies
 - > Rules
 - > Triggers
- > libreria_libro
 - > Columns (11)
 - > Constraints
 - > Indexes
 - > RLS Policies
 - > Rules
 - > Triggers
- > mail_activity
- > mail_activity_rel
- > mail_activity_type
- > mail_activity_type_mail_template_re
- > mail_alias
- > mail_blacklist
- > mail_channel
- > mail_channel_mail_wizard_invite_re

Properties SQL Statistics Dependencies **Dependents**

Type	Name	Restriction
1.3 Sequence	public.libreria_categoria_id_seq	auto
Function	nextval('libreria_categoria_id_seq'::regclass)	auto
Foreign Key	public.libreria_categoria.libreria_categoria_create_uid_fkey	auto
Foreign Key	public.libreria_categoria.libreria_categoria_write_uid_fkey	auto
Primary Key	public.libreria_categoria_pkey	auto

5. Creación de vistas

En este apartado se van a crear las vistas que van a trabajar sobre los modelos del apartado anterior. Las vistas están almacenadas en el directorio **views**, siendo el archivo **views.xml** el que contendrá las vistas correspondientes a nuestro módulo. Se pueden meter varias vistas dentro de un fichero .xml, o bien separarlas en varios ficheros.

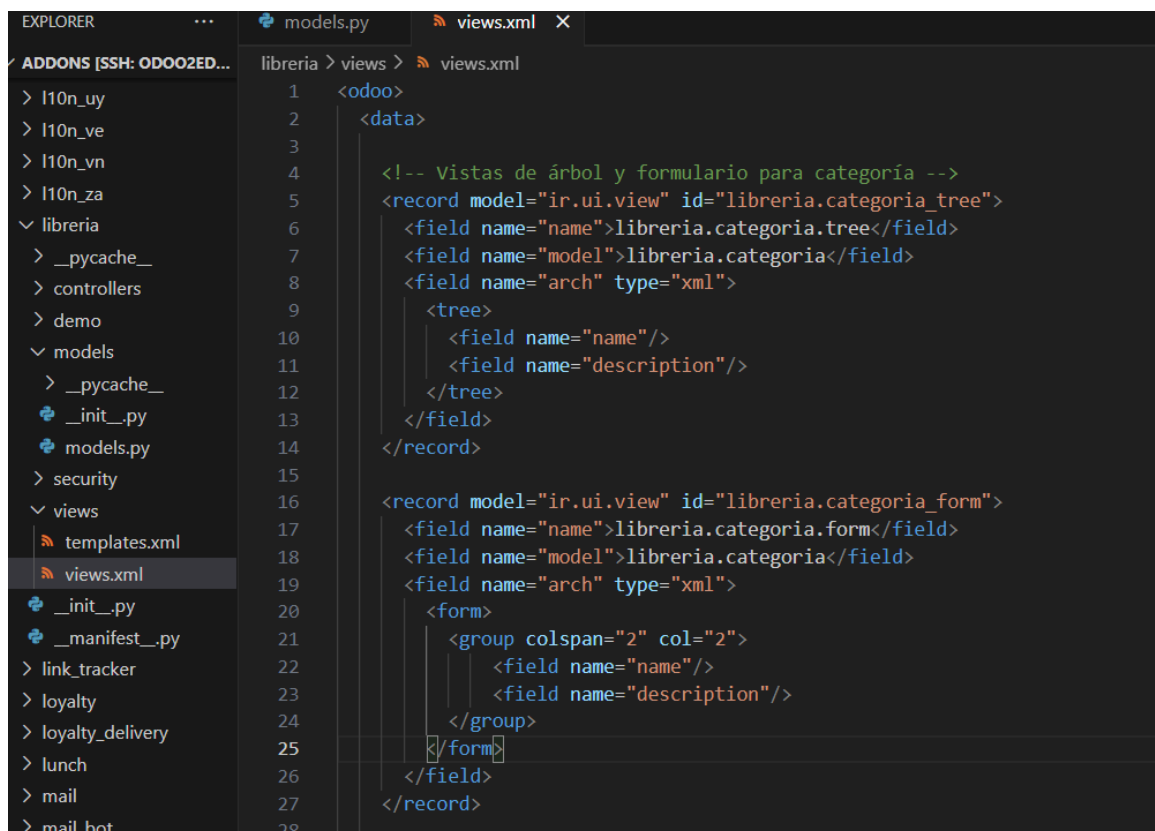
Vamos a crear vistas de tipo árbol y de tipo formulario para cada objeto (categoria y libro), así como las acciones de ventana y los menús para poder ver estas vistas.

5.1. Vistas de tipo árbol y formulario

Pasos para crear vistas de tipo árbol y formulario para cada objeto:

- Todas las vistas se representan con un objeto de tipo **record**, con el tipo de modelo **ir.ui.view**, y tienen un id, el cual **únicamente puede tener un punto**. El id suele llamarse como *nombremodulo.objeto_tipodevista*.
- Como nombre de la vista, *nombremodulo.nombreobjeto_tipodevista*.
- El modelo debe coincidir con el dado en el archivo models.py.
- Si la vista es de tipo árbol se indica con **<tree>**, y si es de tipo formulario, con **<form>**.
- En las vistas de tipo formulario, se recomienda crear **grupos** mediante **<group>**, añadiendo dos columnas para que quede etiqueta-campo, aunque podrían ponerse más, según como quiera maquetar el programador.

Así, la definición de ambas vistas para el objeto *categoria* queda así:



```
1 <odoo>
2 <data>
3
4 <!-- Vistas de árbol y formulario para categoría -->
5 <record model="ir.ui.view" id="libreria.categoria_tree">
6   <field name="name">libreria.categoria.tree</field>
7   <field name="model">libreria.categoria</field>
8   <field name="arch" type="xml">
9     <tree>
10       <field name="name"/>
11       <field name="description"/>
12     </tree>
13   </field>
14 </record>
15
16 <record model="ir.ui.view" id="libreria.categoria_form">
17   <field name="name">libreria.categoria.form</field>
18   <field name="model">libreria.categoria</field>
19   <field name="arch" type="xml">
20     <form>
21       <group colspan="2" col="2">
22         <field name="name"/>
23         <field name="description"/>
24       </group>
25     </form>
26   </field>
27 </record>
28
```

De igual forma se haría para el objeto libro, cambiando los atributos de la vista y los campos del

objeto, siempre teniendo en cuenta lo que queramos ver según lo definido en el archivo de modelos `models.py`. En este caso mostraremos únicamente los campos *name*, *precio*, *ejemplares* para la vista de tipo árbol, y todos los campos para la vista de tipo formulario. Quedaría:

```

29 <!-- Vistas de árbol y formulario para libro -->
30 <record model="ir.ui.view" id="libreria.libro_tree">
31   <field name="name">libreria.libro.tree</field>
32   <field name="model">libreria.libro</field>
33   <field name="arch" type="xml">
34     <tree>
35       <field name="name"/>
36       <field name="precio"/>
37       <field name="ejemplares"/>
38     </tree>
39   </field>
40 </record>
41
42 <record model="ir.ui.view" id="libreria.libro_form">
43   <field name="name">libreria.libro.form</field>
44   <field name="model">libreria.libro</field>
45   <field name="arch" type="xml">
46     <form>
47       <group colspan="2" col="2">
48         <field name="name"/>
49         <field name="precio"/>
50         <field name="ejemplares"/>
51         <field name="fecha"/>
52         <field name="segmano"/>
53         <field name="estado"/>
54       </group>
55     </form>
56   </field>
57 </record>

```

5.2. Acciones de ventana

Se definen en el mismo archivo `views.xml`, y podemos partir de los propios ejemplos existentes en el archivo cambiando los datos. Hay que crear dos acciones de ventana, una para categoría y otra para libro, quedando como sigue:

```

73
74 <!-- actions opening views on models -->
75
76 <record model="ir.actions.act_window" id="libreria.categoria.action_window">
77   <field name="name">libreria.categoria.action_window</field>
78   <field name="res_model">libreria.categoria</field>
79   <field name="view_mode">tree,form</field>
80 </record>
81
82 <record model="ir.actions.act_window" id="libreria.libro.action_window">
83   <field name="name">libreria.libro.action_window</field>
84   <field name="res_model">libreria.libro</field>
85   <field name="view_mode">tree,form</field>
86 </record>
87

```

5.3. Elementos de menú

Faltarían por crear los elementos de menú que nos sirven para ir a las acciones de ventana anteriores. Y de igual forma que antes, nos apoyamos en el ejemplo que viene en el archivo.

El menú general, que no tendrá padre, se llamará **libreria**, y luego dos menús, uno para ir a categorías y otro para ir a libros.

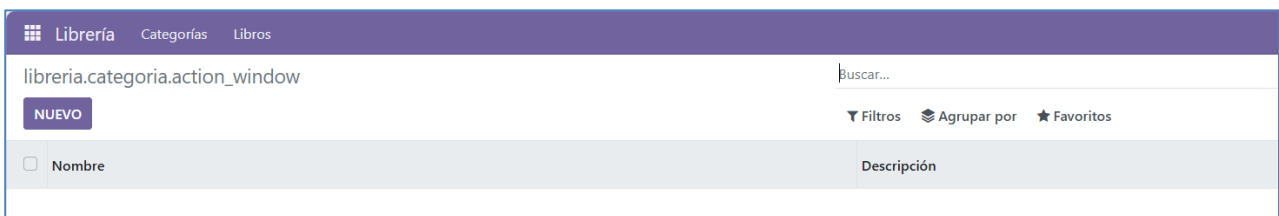
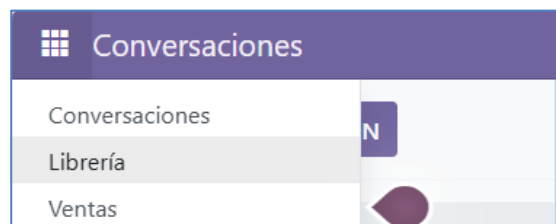
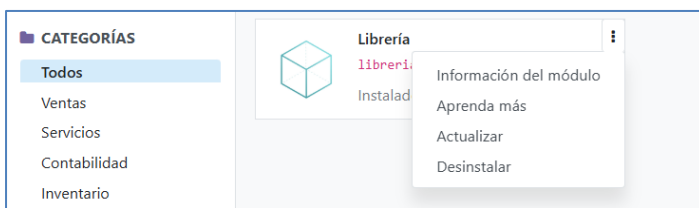
```

105  <!-- Elementos de menu -->
106
107  <menuitem name="Librería" id="libreria.menu_root"/>
108
109  <menuitem name="Categorías" id="libreria.categoria_menu" parent="libreria.menu_root" action="libreria.categoria_action_window"/>
110  <menuitem name="Libros" id="libreria.libro_menu" parent="libreria.menu_root" action="libreria.libro_action_window"/>
111

```

Una vez creadas las vistas, las acciones de ventana y los elementos de menú, todo ello en el archivo **views.xml**, se puede comprobar que efectivamente está en el servidor como corresponde. Para ello basta con situarse sobre la ruta del archivo en cuestión (`/home/odoo/odoo/addons/libreria/views`) y comprobar que está grabado, mediante `ls -lht`, o con `cat views.xml | more`.

Si ahora se va al Odoo y después de actualizar la aplicación se comprueba que se ha instalado correctamente la Librería.



6. Creación de relaciones

Generalmente los modelos que creamos tendrán relaciones entre sí o con otros objetos del sistema.

Las relaciones que podemos crear en Odoo son:

- one2many
- many2one
- many2many

Las tres son análogas a las relaciones SQL que ya conocéis del mundo de las bases de datos. Es importante destacar que para crear una relación one2many en un modelo debemos crear la relación many2one en el modelo con el que establecemos la relación. Por otra parte, es vital que los modelos tengan un campo *name* para que las relaciones funcionen.

En este apartado vamos a programar las relaciones entre los dos objetos que tenemos en nuestro módulo (*libro* y *categoría*). Queremos que un libro pertenezca a una sola categoría y que una categoría pueda tener varios libros, es decir, es una relación que si la miramos desde el lado de categoría es 1 a muchos (1 categoría a muchos libros) y si lo miramos desde el lado del libro es muchos a uno. Se define la relación tanto en categoría como en libro, para poder trabajar con la relación desde los dos sitios.

- Se comienza por el lado del muchos a 1, definiendo en primer lugar la categoría del libro, que solamente habrá una. Se deben especificar los siguientes campos:
 - El objeto contra el que nos estamos relacionando (*libreria.categoría*).
 - El campo que va a salir en la vista (*Categoría*).
 - Si es o no requerido. En este caso sí, es decir, un libro siempre va a tener una categoría.
 - Indicar qué pasa si se borra una categoría. Si se borrara sin más, podrían quedar libros sin categoría, luego parece lógico imponer que si se borra una categoría, se borren en cascada todos los libros que pertenezcan a esa categoría.

Hay que recordar que todos estos valores son idénticos a los de una base de datos relacional. Por tanto, la relación de muchos a uno (libro a categoría), especificada en la clase del libro (*libreria_libro*, en el archivo **models.py**) quedaría así:

```
11 class libreria_libro(models.Model):
12     _name = "libreria.libro"
13
14     name = fields.Char(string="Título", required=True)
15     precio = fields.Float(string="Precio")
16     ejemplares = fields.Integer(string="Ejemplares")
17     fecha = fields.Date(string="Fecha de compra")
18     segmano = fields.Boolean(string="Segunda mano")
19     estado = fields.Selection([('0', 'Bueno'), ('1', 'Regular'), ('2', 'Malo')], string="Estado", default="0")
20     categoria = fields.Many2One("libreria.categoria", string="Categoría", required=True, ondelete="cascade")
```

Hay que destacar que cuando se definen las relaciones *many2one* no se indica ningún campo, únicamente se indica el nombre del objeto. Esto es así ya que por defecto se considera el campo *name*, de ahí que sea necesario que los modelos tengan un campo con ese nombre para que las relaciones funcionen.

Ahora se define la relación inversa, es decir, de categoría a libro.

```
5 class libreria_categoria(models.Model):
6     _name = "libreria.categoria"
7
8     name = fields.Char(string="Nombre", required=True, help="Introduce el nombre de la categoría")
9     description = fields.Text(string="Descripción")
10     libros = fields.One2Many("libreria.libro", "categoria", string="Libros")
```

Cabe destacar que aquí sí es necesario indicar contra qué campo vamos, que será el que tiene el *many2one* en la otra clase (el campo *categoria* creado antes).

Faltaría por último cambiar las vistas, modificando en este caso únicamente las de tipo formulario (archivo *views.xml*), aunque también se podrían modificar las vistas de tipo árbol si se quisiera.

```
<record model="ir.ui.view" id="libreria.libro_form">
  <field name="name">libreria.libro.form</field>
  <field name="model">libreria.libro</field>
  <field name="arch" type="xml">
    <form>
      <group colspan="2" col="2">
        <field name="name"/>
        <field name="precio"/>
        <field name="ejemplares"/>
        <field name="fecha"/>
        <field name="segmano"/>
        <field name="estado"/>
        <field name="categoria"/>
      </group>
    </form>
  </field>
</record>
```

Si se empieza por la vista de libro, se debe mostrar cuál es la categoría del libro, simplemente añadiendo el nuevo campo *categoria* en la vista correspondiente.

Por otra parte, en la vista formulario de la categoría hay que hacer que aparezcan los datos de la categoría y también una tabla con los libros que pertenecen a esa categoría.

En este caso, se crea otra vista de tipo árbol dentro del nuevo campo, indicando que se muestren en ella los campos que se quiera del objeto libro.

```
<record model="ir.ui.view" id="libreria.categoria_form">
  <field name="name">libreria.categoria.form</field>
  <field name="model">libreria.categoria</field>
  <field name="arch" type="xml">
    <form>
      <group colspan="2" col="2">
        <field name="name"/>
        <field name="description"/>
        <field name="libros">
          <tree>
            <field name="name"/>
            <field name="precio"/>
            <field name="ejemplares"/>
          </tree>
        </field>
      </group>
    </form>
  </field>
</record>
```

Ahora, después de reiniciar el servicio de Odoo (ya que se modificó el archivo *models.py*) y de desinstalar y volver a instalar la aplicación *Librería* (o de actualizarla si no se había metido nada) ya se puede ver que la nueva entidad *Librería* está operativa y con todo enlazado correctamente.

7. Creación de vistas de búsqueda

Las vistas de búsqueda permiten redefinir la forma en la que buscamos en Odoo. Tenemos dos opciones a la hora de realizar una vista de búsqueda:

- Campos de búsqueda: simplemente son campos del modelo por los que es posible buscar.
- Filtros: son búsquedas ya definidas con un criterio definido por nosotros.

Se va a ver en primer lugar cómo crear una vista de búsqueda por campos de búsqueda. Cabe señalar que Odoo ya permite una búsqueda por defecto por el campo *name*. Para desarrollar ahora una vista de búsqueda que permita buscar por otro campo, se implementa de nuevo en el archivo **views.xml** y podemos utilizar de apoyo una de las vistas de tipo formulario, pero indicando que ahora la vista será de tipo **search**. Dentro de este **search** es donde se indica por qué campos se va a permitir buscar el objeto. En este caso, implementando una vista de búsqueda para el objeto libro que permita buscar por *name* (Título) y *categoría*, quedaría:

```
<record model="ir.ui.view" id="libreria.libro_search">
  <field name="name">libreria.libro.search</field>
  <field name="model">libreria.libro</field>
  <field name="arch" type="xml">
    <search>
      <field name="name" string="Título"/>
      <field name="categoria" string="Categoría"/>
    </search>
  </field>
</record>
```

Se puede probar actualizando de nuevo la aplicación *Librería*, metiendo libros y categorías de prueba, y comprobando que permite buscar un libro por título y por categoría, permitiendo también buscar por varias cosas a la vez.

Librería Categorías Libros			Administrator (Cinco)		
libreria.libro.action_window			Buscar...		
NUEVO			Filtros Agrupar por Favoritos 1-5 / 5		
☐ Título	Precio	Ejemplares			
☐ Cien años de soledad	15,00	6			
☐ El coronel no tiene quién le escriba	12,50	2			
☐ La amenaza de Andrómeda	11,20	2			
☐ Dune	12,00	3			
☐ El problema de los tres cuerpos	17,45	0			

Librería

Categorías

Libros

A

Administrator (Cinco)

libreria.libro.action_window

Buscar...

NUEVO

Filtros

Agrupar por

Favoritos

1-5 / 5

☐	Título	Precio	Ejemplares
☐	Cien años de soledad	15,00	6
☐	El coronel no tiene quién le escriba	12,50	2
☐	La amenaza de Andrómeda	11,20	2
☐	Dune	12,00	3
☐	El problema de los tres cuerpos	17,45	0

Como se indicó al principio de este apartado, el otro tipo de vista de búsqueda sería mediante filtros. Esto se desarrolla mediante la etiqueta **filter**. Además, se debe especificar la condición lógica mediante la que se va a filtrar, en este caso a través de la etiqueta **domain**. La especificación del dominio presenta tres partes:

- Primera: campo sobre el que se va a escribir la condición.
- Segunda: establecer la condición.
- Tercera: valor sobre el que se compara.

Para el caso concreto que se está analizando, vamos a suponer que se hace un filtro de búsqueda para libros de menos de 15 €. Con esto, la vista de búsqueda definitiva quedaría:

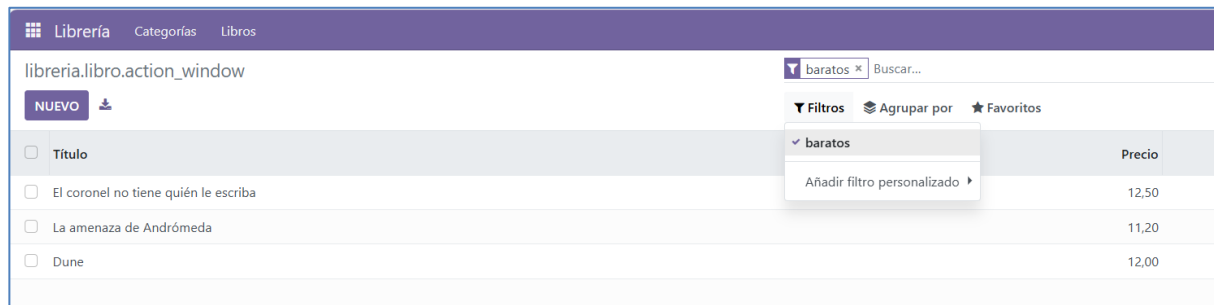
```
<record model="ir.ui.view" id="libreria.libro_search">
  <field name="name">libreria.libro.search</field>
  <field name="model">libreria.libro</field>
  <field name="arch" type="xml">
    <search>
      <field name="name" string="Título"/>
      <field name="categoria" string="Categoría"/>
      <filter name="baratos" domain="(['precio', '<;', 15)]"/>
    </search>
  </field>
</record>
```

Los principales operadores de comparación en XML se agrupan en la siguiente tabla:

Descripción	PY	XML
Igual	=	=
Distinto	!=	<>
Mayor que	>	>
Mayor o igual que	>=	>=

Menor que	<	<
Menor o igual que	<=	<=

Como siempre que se modifica un archivo .xml, actualizar la aplicación y probarla.



8. Creación de campos calculados

Un campo calculado es un campo que se obtiene a partir de los campos simples de un objeto. Sería un campo que, por definición, aunque aparezca en la vista, no se puede tocar, pero que se actualiza automáticamente al modificar alguno de los campos de los que depende.

Para desarrollar un campo calculado se trabaja en el archivo **models.py**, donde se debe escribir el código para que el campo sea capaz de realizar el cálculo correspondiente, a través de un **método** del objeto.

La manera de definir un campo calculado en una clase es idéntica a un campo normal, con la particularidad de que se va a usar la palabra reservada **compute** para llamar a la función privada (el nombre con un solo guión bajo) en la que se va a insertar el código para realizar el cálculo. La palabra **store** se utiliza para indicar que el campo se calcule únicamente cuando se modifican los valores de los que depende, de manera que no se esté calculando continuamente.

Siguiendo con el ejemplo de Librería, un campo calculado podría ser *importetotal*, entendiendo este campo como la multiplicación del precio de un libro por el número de ejemplares disponibles. Tanto el campo como el método correspondiente se definen dentro de la clase *libreria_libro*.


```
class libreria_libro(models.Model):
    _name = "libreria.libro"

    name = fields.Char(string="Título", required=True)
    precio = fields.Float(string="Precio")
    ejemplares = fields.Integer(string="Ejemplares")
    fecha = fields.Date(string="Fecha de compra")
    segmano = fields.Boolean(string="Segunda mano")
    estado = fields.Selection([('0', 'Bueno'), ('1', 'Regular'), ('2', 'Malo')], string="Estado", default="0")
    categoria = fields.Many2one("libreria.categoria", string="Categoría", required=True, ondelete="cascade")
    importetotal = fields.Float(string="Importe total", compute="_importetotal", store=True)

    @api.depends('precio', 'ejemplares')
    def _importetotal(self):
        for r in self:
            r.importetotal = r.ejemplares*r.precio
```

En este código cabe señalar lo siguiente:

- El parámetro **self** representa un parámetro implícito para referenciar al propio objeto, similar al this de Java.
- El bucle **for** se debe poner siempre, ya que este método podría ser llamado por una lista de registros, y no solo por uno, de manera que se itera sobre lo que le llega a través del parámetro self, que serán siempre registros del tipo de la clase, en este caso del tipo *libreria_libro*.
- Faltaría por señalar únicamente que a este método se le añade una **anotación**, que sirve para indicar cuándo debe de llamarse, a través de **@api.depends**. En este caso se especifica de qué valores depende el método y que si cambia *precio* o *ejemplares* debe actualizarse el valor que tiene almacenado.

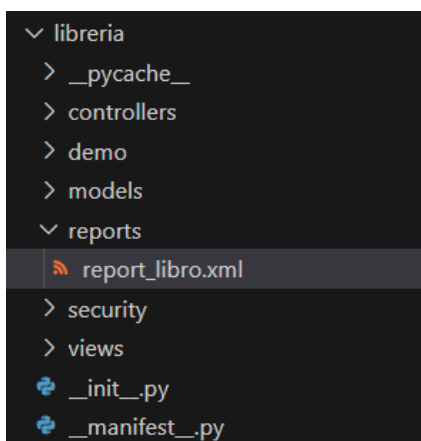
Faltaría por último añadir el nuevo campo en la vista correspondiente (será, lógicamente, en la vista de formulario de libro). Luego se debe reiniciar Odoo, por modificar un archivo Python, y actualizar la aplicación, como siempre que se modifica en archivo .xml, y ya estaría totalmente funcional.

```
<record model="ir.ui.view" id="libreria.libro_form">
  <field name="name">libreria.libro.form</field>
  <field name="model">libreria.libro</field>
  <field name="arch" type="xml">
    <form>
      <group colspan="2" col="2">
        <field name="name"/>
        <field name="precio"/>
        <field name="ejemplares"/>
        <field name="fecha"/>
        <field name="segmano"/>
        <field name="estado"/>
        <field name="categoria"/>
        <field name="importetotal"/>
      </group>
    </form>
  </field>
</record>
```

Librería	
Categorías	Libros
libreria.libro.action_window / Dune	
Título ?	Dune
Precio ?	12,00
Ejemplares ?	3
Fecha de compra ?	29/01/2025
Segunda mano ?	☐
Estado ?	Bueno
Categoría ?	Ciencia ficción
Importe total ?	36,00

9. Creación de informes

Al igual que en la UD4, se pueden crear informes para los objetos en Odoo. Un informe se crea a través del motor de informes Qweb y el resultado suele ser un .pdf o un .html (aunque podría tener otro formato). El lenguaje utilizado para crear los informes es idéntico al utilizado en la UD4.



Siguiendo con el ejemplo de la aplicación *libreria*, lo primero sería crear dentro de nuestro módulo un directorio nuevo, de nombre **reports**, para meter en él cada uno de los informes que se vayan obteniendo. Se le añade uno de prueba (*report_libro.xml*) en el que se van a sacar los datos de un libro.

Todo el informe irá en este archivo, distribuido en dos campos:

- Un campo **report**, que va a llevar la definición del informe. En este campo se indica el id del informe (*id*), sobre qué modelo trabaja el informe (*model*), cómo se va a llamar el informe (*string*) y dos campos (*name* y *file*) que deben coincidir exactamente con lo que se indica debajo, en el template. Por último, un campo *report_type* para indicar el tipo (html, pdf, etc.).
- Un campo **template**, que va a llevar el contenido del informe, es decir, qué va dentro, y debe cumplir una serie de requisitos:

- El *id* del template debe coincidir exactamente con el campo *name* del report, es decir, con la vista especificada en el campo *name*.
- La hoja general del informe (*t-call*) metida dentro de un contenedor (*html_container*).
- Una línea de tipo `<t t-foreach="docs" t-as="libro">` que representa el hecho de que se debe iterar cada uno de los libros que pudieran llegar, utilizando la variable *libro* especificada en *t-as*.
- Una línea de tipo `<t t-call="web.external_layout">` para permitir que el informe vaya maquetado, como por ejemplo la factura que lleva una cabecera con un logo de empresa, el pie de página y demás. Si se quiere que salga una hoja en blanco no se pondría esta línea.
- Dentro de la etiqueta `<div class="page">` se meterían los campos que se quieren mostrar en el informe.

```

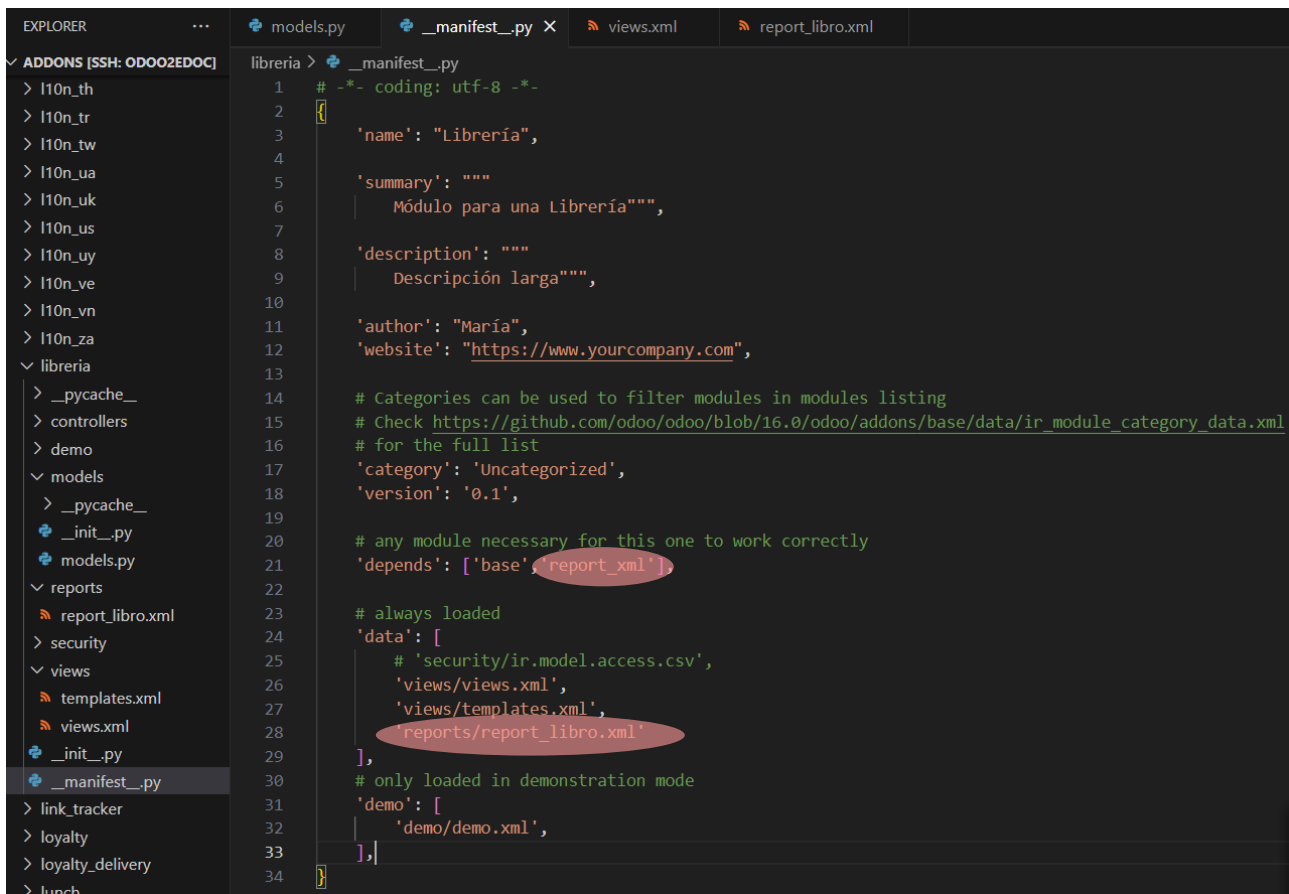
ia > reports > report_libro.xml
<odoo>
  <data>
    <report
      id="libreria.report_libro"
      model="libreria.libro"
      string="Informe libro"
      name="libreria.report_libro_view"
      file="libreria.report_libro_view"
      report_type="qweb-pdf"/>

    <template id="report_libro_view">
      <t t-call="web.html_container">
        <t t-foreach="docs" t-as="libro">
          <t t-call="web.external_layout">
            <div class="page">
              <h2 t-field="libro.name"/>
              <div>
                <strong>Precio:</strong>
                <span t-field="libro.precio"/>
              </div>
              <div>
                <strong>Ejemplares:</strong>
                <span t-field="libro.ejemplares"/>
              </div>
              <div>
                <strong>Categoría:</strong>
                <span t-field="libro.categoria"/>
              </div>
            </t>
          </t>
        </t>
      </template>
    </data>
  </odoo>

```

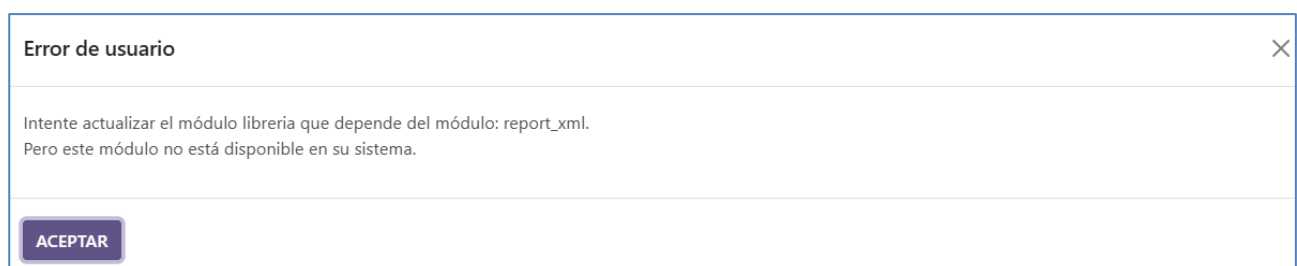
Por último, se debe modificar el archivo `__manifest__.py` para indicar dos cosas:

- Cada vez que se actualiza el módulo, se debe actualizar también el .xml que se acaba de crear (`report_libro.xml`).
- Nuestro módulo también depende del módulo `report_xml`.



```
1  # -*- coding: utf-8 -*-
2  {
3      'name': "Librería",
4
5      'summary': ""
6      |   Módulo para una Librería"",
7
8      'description': ""
9      |   Descripción larga"",
10
11     'author': "María",
12     'website': "https://www.yourcompany.com",
13
14     # Categories can be used to filter modules in modules listing
15     # Check https://github.com/odoo/odoo/blob/16.0/odoo/addons/base/data/ir_module_category_data.xml
16     # for the full list
17     'category': 'Uncategorized',
18     'version': '0.1',
19
20     # any module necessary for this one to work correctly
21     'depends': ['base', 'report_xml'],
22
23     # always loaded
24     'data': [
25         # 'security/ir.model.access.csv',
26         'views/views.xml',
27         'views/templates.xml',
28         'reports/report_libro.xml'
29     ],
30     # only loaded in demonstration mode
31     'demo': [
32         'demo/demo.xml',
33     ],
34 }
```

Ahora, como siempre que se modifica un archivo .py, se reinicia el Odoo, se actualiza la lista de aplicaciones para que considere el nuevo módulo *report_xml*, y se actualiza la aplicación *libreria*. Sin embargo, el módulo *report_xml* no está disponible en el sistema.



Este módulo se debe descargar e instalar manualmente. Se busca de la forma indicada en la unidad didáctica 3, escribiendo en el navegador:

report_xml site: <https://github.com/OCA>

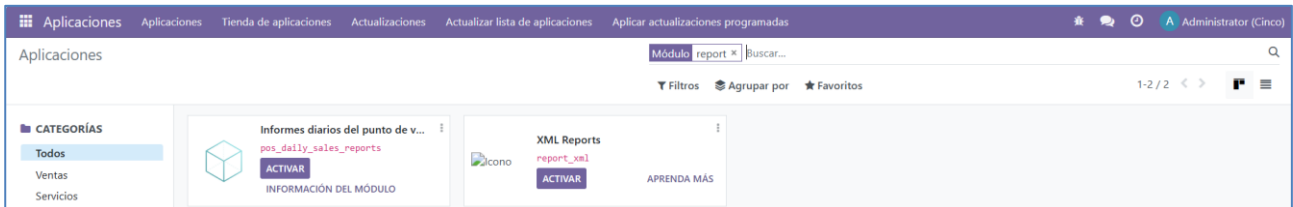
Y se descarga en el servidor.

git clone -b xx.x https://github.com/OCA/reporting-engine.git

su odoo

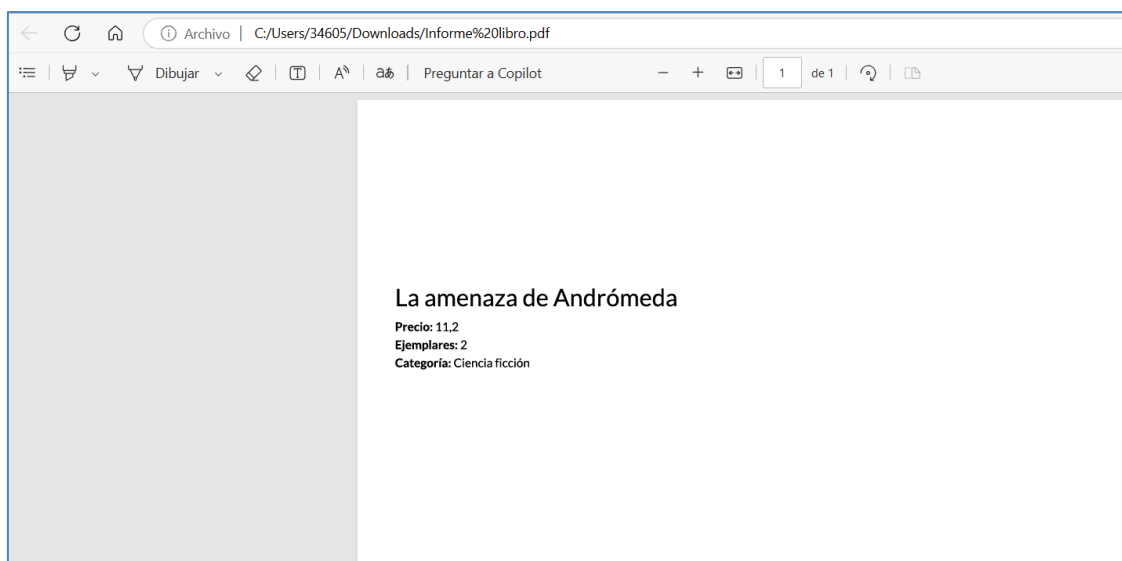
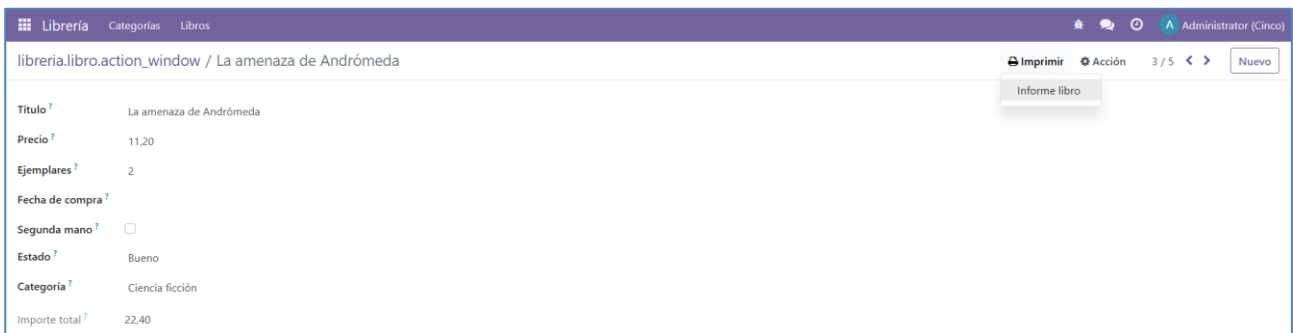
```
cp -r reporting-engine/report_xml /home/odoo/odoo/addons
```

Una vez hecho esto se actualiza la lista de aplicaciones y ya se puede ver que el módulo *report_xml* está disponible para instalar.



Se puede instalar directamente, o actualizar nuestra aplicación *libreria*, con lo cual el módulo *report_xml* ya se instalará en cascada.

Si ahora se va a alguno de los libros creados en la aplicación *libreria*, se puede ver que existe la opción de *imprimir Informe libro*, con lo que se descarga un archivo .pdf con la información correspondiente a ese libro.



10. Configuración de permisos y grupos

Una vez creado un módulo, es interesante dejar creados grupos que gestionen los accesos a las partes de ese módulo. En el caso de la librería, crearemos un único grupo *Librería-Responsable* que tendrá todos los privilegios sobre el módulo *libreria*. De manera análoga, se podrían crear otros grupos que diesen permisos más restrictivos sobre el módulo.

Para ello se deben editar dos archivos: un **security.xml** y un **archivo .csv**, donde se indicará qué permisos tiene cada grupo sobre cada objeto de Odoo individual.

Se desarrolla en primer lugar el archivo **security.xml**. Este archivo se crea dentro de la carpeta **security**, y en él se indica qué roles o qué grupos se van a tener para este módulo, escribiendo un registro (*record*) por cada uno. Cada registro trabajará siempre sobre el modelo **res.groups**. En este caso estableceremos un único grupo identificado como *libreria_manager*.



```
libreria > security > security.xml
1 <odoo>
2   <data>
3     <record id="libreria_manager" model="res.groups">
4       <field name="name">Librería / Responsable</field>
5     </record>
6   </data>
7 </odoo>
```

Una vez creado el grupo, se debe establecer qué permisos tiene sobre las tablas físicas del módulo, es decir, sobre las tablas *libreria_categoria* y *libreria_libro*. Esto se define en el archivo .csv que viene precreado en el directorio security (archivo **ir.model.access.csv**). Este archivo ya viene con una cabecera, correspondiente a la primera línea, que no se toca, y una serie de campos en la segunda línea que se deben modificar teniendo en cuenta la siguiente información:

- Primer campo: id de la regla que se está estableciendo (no puede llevar espacios).
- Segundo campo: nombre de la regla (en este caso puede admitir espacios).
- Tercer campo: se construye con `model_` y la tabla sobre la que se quieren dar los permisos.
- Cuarto campo: grupo sobre el que se van a aplicar los permisos, que debe coincidir con el id que se le ha dado a dicho grupo en el archivo `security.xml`.
- Los últimos cuatro números corresponden a los permisos de lectura, escritura, creación y borrado.

Se agrega un permiso para la categoría y otro para el libro, siguiendo el mismo patrón.

```

ADDONS [SSH: ODOO2EDOC]  libreria > security > ir.model.access.csv
1 id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
2 categoria_responsable,categoria responsable,model_libreria_categoria,libreria_manager,1,1,1,1
3 libro_responsable,libro responsable,model_libreria_libro,libreria_manager,1,1,1,1

```

Con esto lo que se está haciendo es dar todos los permisos al grupo *libreria_manager* sobre libro y sobre categoría.

Una vez hecho todo esto faltarían un par de detalles para que todo funcione correctamente:

- El archivo **security.xml** creado se debe añadir en el **__manifest__.py**, para que se cargue, así como el **ir.model.access.csv**, ya que se ha modificado.

```

security 23
ir.model.access.csv 24
security.xml 25
views 26
__init__.py 27
__manifest__.py 28
link_tracker 30

```

```

# always loaded
'data': [
    'security/security.xml',
    'security/ir.model.access.csv',
    'views/views.xml',
    'views/templates.xml',
    'reports/report_libro.xml'
],

```

Decir también que aquí los archivos se cargan secuencialmente, luego es mucho mejor poner lo referente a security antes de las vistas, ya que se utiliza su contenido en views.xml.

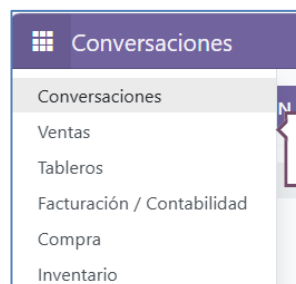
- En los elementos de menú (archivo *views.xml*) se debe indicar que solo muestre este menú para el grupo que se acaba de crear (*libreria_manager*).

```

126 <!-- Elementos de menú -->
127
128 <menuitem name="Librería" id="libreria.menu_root" groups="libreria_manager"/>
129
130 <menuitem name="Categorías" id="libreria.categoria_menu" parent="libreria.menu_root" groups="libreria_manager" action="libreria.categoria_a
131 <menuitem name="Libros" id="libreria.libro_menu" parent="libreria.menu_root" groups="libreria_manager" action="libreria.libro_action_window
132

```

Se reinicia Odoo, se actualiza la aplicación y ya se puede probar. Como es lógico, se verá que la aplicación *libreria* no está disponible, ya que por defecto el usuario con el que se está conectado no tiene permisos sobre el grupo Librería / Responsable. Si se le dan permisos para ese grupo y se refresca la aplicación (F5) se puede comprobar que la aplicación Librería vuelve a estar disponible y totalmente operativa.



Usuarios / Administrator

Mostrar número de lote y serie en albaranes de entrega [?]

☐

Mostrar incoterms en los Pedidos de Ventas y en facturas relacionadas [?]

☐

Editor de plantillas de correo [?]

☒

Gestionar números de lote/serie [?]

☐

Gestionar múltiples unidades de medida [?]

☐

Gestionar paquetes [?]

☐

Administrar variantes de productos [?]

☐

Gestionar categorías de almacenamiento [?]

☐

Facturas proforma [?]

☐

Plantillas de presupuesto [?]

☐

Recibo de venta [?]

☐

Impuesto B2B [?]

☒

Utilizar informe de recepción [?]

☐

EXTRA RIGHTS

Creación de contactos [?]

☒

Multimoneda [?]

☐

OTHER

Evitar la depuración de campos HTML [?]

☒

Mostrar el número de serie y lote en las facturas [?]

☐

Bloquear pedidos confirmados [?]

☐

Gestionar diferentes propietarios de stock [?]

☐

Gestionar múltiples ubicaciones de almacén [?]

☐

Gestionar múltiples almacenes [?]

☐

Gestionar el embalaje del producto [?]

☐

Gestionar flujos de inventario push y pull (jalar & empujar) [?]

☐

Imprimir un código de barras GS1 para números de serie y de lote [?]

☐

Recibo de compra [?]

☐

Pida una firma en sus órdenes de entrega [?]

☐

Envíe un correo electrónico de recordatorio automático para confirmar la entrega [?]

☒

Impuesto B2C [?]

☐

Utilizar picking por oleadas [?]

☐

Múltiples compañías [?]

☐

Librería / Responsable [?]

☒

Acción

11. Cuestiones finales

Se va a ver para terminar cómo establecer un icono para el nuevo módulo y cómo precargar datos por defecto. De esta forma, al buscar la aplicación aparece su icono asociado, y al precargar datos, en este caso se cargarán un par de categorías, el usuario ya puede empezar a trabajar normalmente con el nuevo módulo teniendo algún dato de inicio.

11.1 Establecer icono

Para establecer un icono, de tipo .png, se tiene que subir ese icono al servidor. Para hacer esto podemos usar el propio VSC (arrastrar y soltar el archivo) y también el cliente **WinSCP** ya visto en la unidad didáctica 2.

Como es lógico, este icono (de nombre **icon.png**) se debe subir al módulo *libreria*, dentro de un directorio llamado **description**, que a su vez está en otro llamado **static**. Ambos directorios deben ser creados, pudiendo hacerse desde VSC o desde el cliente WinSCP (dando previamente permisos de escritura al directorio *libreria*) o desde el propio servidor.

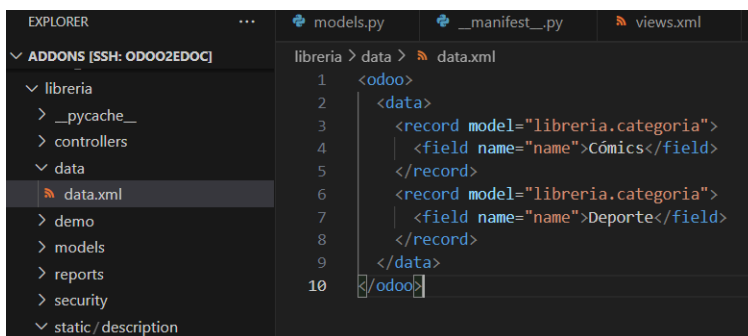


Y con esto ya se visualizaría el icono para ese módulo. Solamente hay que reiniciar el servidor y actualizar la aplicación.



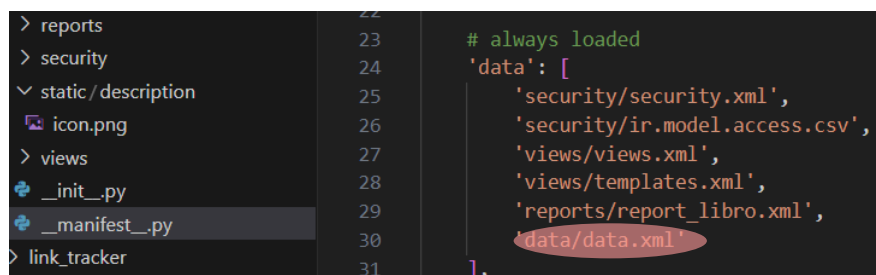
11.2. Datos precargados

Para los datos precargados se debe crear un nuevo directorio **data**, y dentro, un archivo llamado **data.xml**.



Este archivo tendrá la misma estructura que todos los archivos de Odoo, conteniendo un registro por cada fila que se quiera meter en cualquiera de las tablas. En este ejemplo metemos dos filas en la tabla de categorías, una de cómics y otra de deporte.

Faltaría añadir este archivo en el **__manifest__.py**, pudiendo ir en este caso al final



Finalmente se reinicia el servidor y se desinstala y se vuelve a instalar la aplicación para probarlo. En este caso, no vale con actualizar la aplicación, ya que se trata de establecer datos precargados, es decir, datos que aparecerán por defecto, y si el sistema detecta que la aplicación ya tiene datos metidos, falla. Se debe recordar al reinstalar que hay que darle al usuario permisos sobre el grupo Librería / Responsable, según se indicó en el punto anterior.

